NPTEL

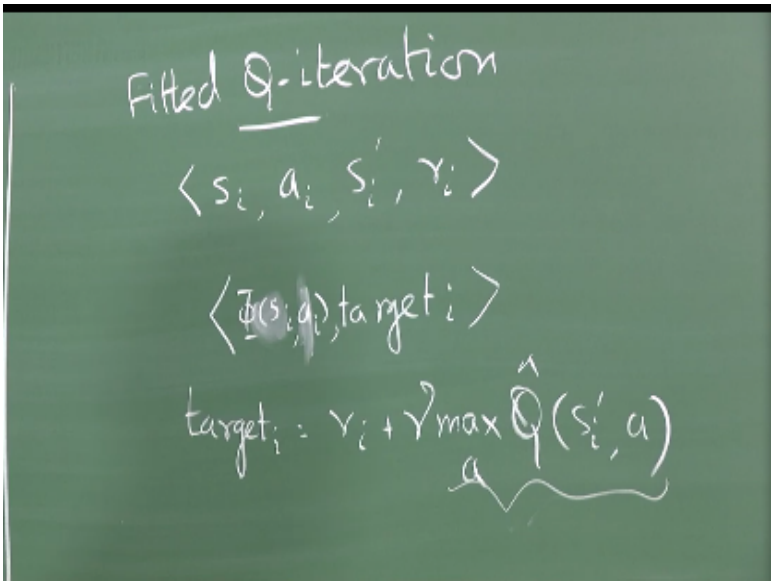
NPTEL ONLINE COURSE

REINFORCEMENT LEARNING

DQN and Fitted Q-Iteration

Prof. Balaraman Ravindran
Department of Computer Science and Engineering
Indian Institute of Technology Madras

(Refer Slide Time: 00:15)



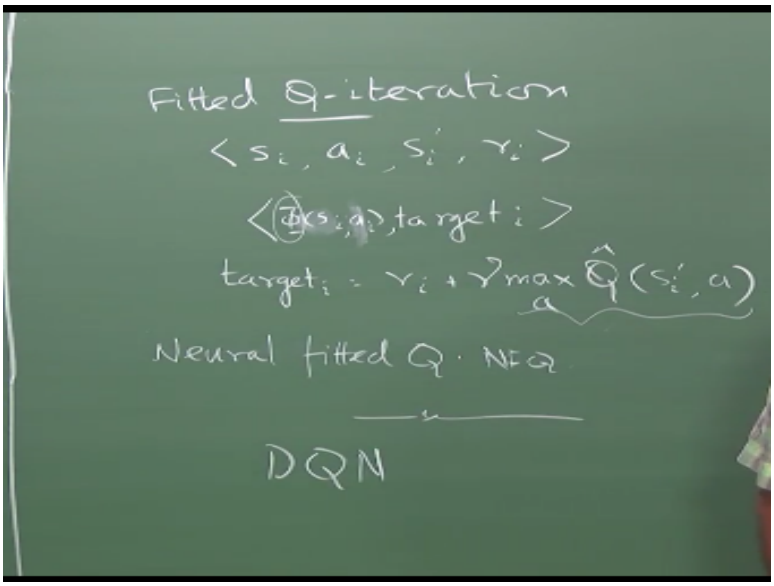
So the next thing I started talking little bit about fitted Q iteration so the idea behind fitted Q iteration is very simple right so essentially what you do is you gather data of the form right, so this is exactly what you do in LSTD right and what do you do in fitted Q iteration, do what, yeah so essentially you learn the thing so I have my Si , target i, this is not really this is not really be Si, it will be right.

Is it correct, is there something wrong, right and then what is target i, right so this is the target so I form a set of tuples like this feed it to the function approximator right and then it will train something it will spit out a new Q hat right and then I go back I do the same thing again right so what is the requirement that I should have that I should be able to run this max right so that essentially means that I should have taken that all state action pairs right.

I mean otherwise I really have no meaningful data on which I am estimating that max right or I have taken at least similar looking state action pair so that I can use my function approximator in order to generate the values for those right so the static training data that I work with right the static training data I work with should be broad enough right should be representative enough so that I sample all kinds of state action combinations at least modulo my φ.

Right so what do I mean modulo my φ, when I look at the φ of s , a, I should have sampled enough of those representations right I need not necessarily sample every state action pair right but I should have sampled it enough so that across the φs I have sampled all possible state action combinations I will see some amount so that the generalization is useful okay so this is essentially fitted Q iteration right.

(Refer Slide Time: 03:30)



One very popular version of fitted Q iteration is essentially neural fitted Q or sometimes called NFQ so what is what do you think neural fitted Q does, exactly right you keep it you make this make these datasets right you make this data set, fit it to a neural network right and then the neural network chugs away and then it gives you back a new Q hat represented by the neural network and then you create another data set and you keep giving this to this until it converges okay.

So what are the problems that you might face when you are doing this one I already told you, what, what is the, I mean I discussed one problem already with you what is the problem yes exactly so you should make sure that you should have enough samples in the data right so that is one that is one problem right another problem is in general for any function approximation what is the other problem let us assume that I have a way of doing max on this yeah of course max is a problem.

But suppose you have small number of discrete actions right I can train one network for each and so I can just evaluate each one of those neural networks and then figure out which is the max right, so if you have small number of discrete actions you can get away with that, you remember in the very beginning I was telling you for representing Q you have multiple options.

So one of them was to have different networks or different function approximators for each action and in this case you will have different neural networks for each action but what is the other problem that you could encounter right this is not something peculiar to what I tell, told today or yesterday it is a problem with all kinds of function approximation, which is what we have been talking about for the last two days.

Whatever it is your favorite training algorithm you use it right I will give you training data right it is a regression problem solve it using whatever is your favorite regressor right, why not, it takes the state as an input gives Q s a as the output, you should revise from previous classes since this is one of the options I gave for doing control with function approximation right in fact I indexed the φ with the a as a subscript, right.

I said φ a of s is something that you can use that is one different feature sets you could use for different actions as well and so on so forth and then I also had a θ a and stuff we looked at all of that what is the problem, Jana knows the problem, what is the problem, oh come on you know that, tsk, yeah but I am not solving a non-stationary regression problem here right, that it is the nice thing about all of this fitted Q business.

So every time I solve a regression problem I am solving a stationary problem, right whether it be LSTD or fitted Q so every time I am solving a regression problem I am solving a stationary regression problem so it is not like the online updates that we were doing where things were non-stationary, so it becomes a little hairy but now I am every time I do it is actually a fixed data set on which I am fitting.

Yeah we do, this is assuming that the cost is cheap, no no the experience we are not generating again but this φ target pair we are generating every time we go around right that is what he meant, oh my god, exactly, my god, it took you guys so long to get there right I told you what target is but who gives you the φ right so we spoke about some very simple ways of generating φ assuming things are going to be linear function approximators bla bla bla and things like that right.

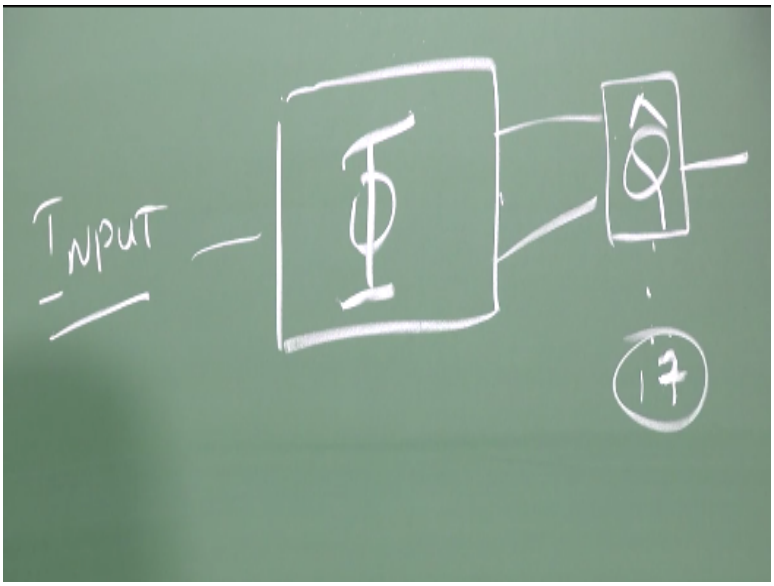
But then how do you come up with φs, whatever nowadays people do not care about that right now a days you know what they do they throw a deep neural network at it they do not throw a normal neural network at it they throw a deep neural network at it right and then it learns learns stuff, have you seen any convergence proof for DQN, no no people do not show convergence.

They just dazzle you by organizing a public match with the worlds champion go champion and things like that, you do not ask them oh by the way can you show me that alpha go converges right it just beats the world champion you got do not ask them for convergence right so that is basically it so the proof of the pudding is in the eating in these cases right essentially it just works, so I did not mean to imitate him or anything just it was subconscious right.

So DQN is a deep Q network that addresses a few of the problems that we spoke about, the first one is it does not use a offline set of samples it actually samples online right so then that in some sense allows you to not have to worry about the restrictivity of using a offline set of samples right the second thing it helps you solve is the φ question, so I am not going to tell you how it solves the φ question.

Because it requires me to tell you about deep networks requires me to tell you about convolutional networks and so on so forth right so let us ignore that for the time being let us say that I give you an input right.

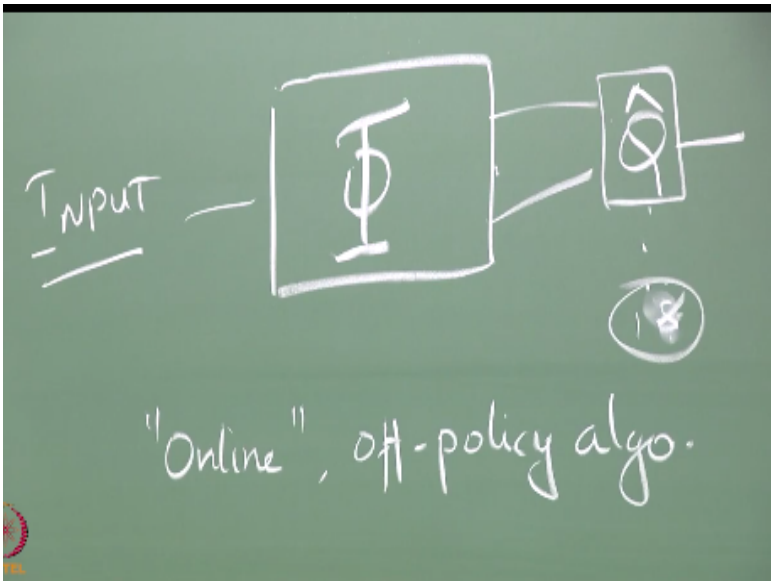
(Refer Slide Time: 11:00)



I give you an input, it goes through a black box okay that computes the φ and I get the φ at the other side and then it goes through a Q hat network and I get this right, so in this case they basically have so the DQN is the is the network that was trained to play the Atari games right remember I showed you Atari games in the very beginning so DQN is the network that is trained to play Atari games.

So in Atari games you have 17 actions, so you have eight directions in which you can move your joystick and then there is a fire button right and you can move the joystick in 8 directions with the fire button pressed or without the fire button right so that gives you 16 and the 17th action is no op, right do not do anything right, so that is the seventeenth action so there are 17 actions so you basically you have seventeen such seventeen such networks here right.

(Refer Slide Time: 12:28)



Eighteen is it, what is the eighteenth action, just fire and do not do anything, I thought it was seventeen, are you sure it is eighteen, okay they say it is eighteen, it is eighteen yeah I am quite sure then the eighteenth action must be do not move the joystick but just press the fire button okay that there has to be an action for that right so yeah so 18 actions so there are 18 such networks and so we get 18 outputs at the end of it right this will give you all the Q values right.

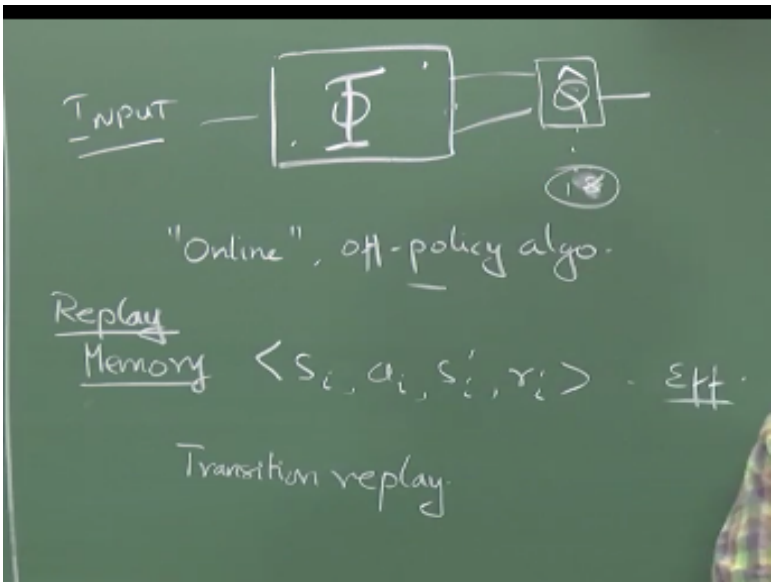
Now you can just go pick your best Q right the best action according to the Q function perform that right and then see what happens to the state it will go to a next state automatically and then what you do you feed that next state into the φ box right and then you find the corresponding action right and then use that for your Q update right just do it in one shot right so you do not actually do this kind of target generation.

And then have a whole set of things and then do this you are actually doing online updates right as and when you see a transition you do this updates right but there are problems in doing this right so you could potentially run this as a, right you could potentially run this as a online but off policy algorithm because Q learning is off policy right you are learning about the optimal policy while you are doing some behavioral policy and you could also run it as an online algorithm.

Because we are going to be updating along the trajectory that you are seeing right so I am not writing down any equations all of you know what the Q learning equation is right so essentially you do the Q learning right that is possible there are some problems in doing this online updates for this so what do you think are the problems, problems in doing an online Q learning, variance, yes can you elaborate on that, bunch of things that can go wrong so the first thing is well if you go by if you go by what the what the paper the DQN paper tells you the first disadvantage they point out is that you are not using the samples efficiently right.

I am just taking and drawing one sample right and then I am just using it for one update and then I throw it away right, correct, is there some way of fixing that, batch mode, essentially what you do is.

(Refer Slide Time: 15:13)



You basically have a memory right and you add your, to that memory, right so as and when I get an interaction here in the world I keep adding it to the memory right and then I keep pulling these things out of memory once in a while and then I replay them, right so I do not necessarily need to have actually seen the state again but then I will just pull it out of my memory and I will replay them.

So therefore this memory is sometimes called replay memory okay so replay memories are not something which the DQN guys introduced it actually goes all the way back to 93 there is this guy called Longji Lin right who did bunch of it actually interesting stuff with RL so one of the things that Lin did was this experience replay, so what Lin's experience replay was is that you remembered entire trajectories like this right.

And you just pull these trajectories out of your memory periodically and then reapply those trajectories right so from the start to the beginning right it is just like you are running over the same thing again and again in your head you know it is like reliving your life experience so you just did something yesterday and then you just think over it in your head and then you say okay that is what I did.

So let me let me memorize it or something like that you kind of replay your experience okay so I am not being facetious here and it is actually seriously as one of the things if you think about it

humans actually do a lot of replay in your head, you think about what you did right so this is why people play a shot again after they have played it and think like that you kind of replay what you are thinking right so this is, it kind of took inspiration from that and then came up with this kind of replay memory right so but in the Lin version of replay memory you replay entire trajectories, there is a difference.

So the way at least so Rich Sutton talks about planning is that you use these experiences that you have right and convert, so go from S A S' R tuples like this you estimate your probability of S' given S , A, you estimate your expected reward given S A S', exactly so this you are not this is like this is like exactly like having the same experience all over again right if you run a simulation that is different.
If you run a simulation that means you are sampling you are building the model and then generating a sample from the model right so in the Lin case you take an existing sample already which you sample from the real world right you do not rebuild the model right you take the sampled world and then you just rerun the same sample again and again right it is like over sampling the same thing again and again right.

So instead of trying to do a conscious estimation of the model parameters right it could very well be the system is very complex right and you do not really know what are the possible outcomes right so you just in some way if you think about it if you if you go through the samples often enough right you can argue that what you are converging to will be something similar to the certainty equivalence estimate for this data that you have gathered right.

But then you need to have some little bit more constraints on to ensure them right so but replay memory is a lot lighter than actually trying to estimate the full model you do not try to do a model for example in the Atari case right your states are continuous right so you are really not fitting a model right so you are really not trying to fit a model you are just replaying the episodes at your place just like.

You play one game and then you just play it again and again and again hundred times yeah, Lin's work no does not sample uniformly he samples according to the trajectories that are the

according to the current policy, Oh from the memory he samples trajectories uniformly he does not sample transitions uniformly yeah samples trajectories uniformly yeah at least that is what I remember the 93 version of the paper did.

He had a follow up version in 96 I do not know if he made any changes to that but 93 version of replay memory that is what it did, it just samples uniformly from whatever is that you just keep throwing them into memory and then the sample trajectories uniformly from them, good point these are all questions to ask right these are all different variants that you can explore so the original DQN thing what they did was they just basically had a memory of the last hundred transitions or so right the original DQN paper.

But then since then there have been many variants that people have played around with where they throw away some of the trajectories so preferentially right earlier it was just based on history right so if you are more than 100 time steps old they throw it away right so now you can start playing around with preferentially retaining more rare transitions in the in memory and things like that yeah yeah you have to cut it at some point, all kinds of questions will happen yeah.

So there are lots of issues that you will have to tackle here right ok so the replay memory allows us to do efficient reuse right so the efficient reuse of samples is taken care of, so what is the next problem that we need to take care of this comes from the fact that we are doing something that is online right so it gives rise to two artifacts one if you think about it my states are changing slowly typically right when I am looking at these games my states are changing slowly right.

And typically my reward also would change slowly right so if you look at successive inputs that I am going to see they will all look very similar because my states would have changed very little you know maybe one enemy moved here and I moved here and the bullet has moved up two pixels right, so the states will look all very similar right therefore the changes the updates that I am going to be making will be very strongly correlated right from one update to the next right will all be very strongly correlated so that can lead to all kinds of problems in the convergence right.

So what they wanted to do was a way of breaking this correlation in fact they had problems with convergence when they were actually doing the updates only along the trajectories right they had problems with convergence because every update was very very similar to each other so it is like taking the same update and then applying it multiple times, right so it is kind of messes up with the, with the gradient following that you need to do right.

So what they did was they said okay we will put everything into replay memory right so every time we get this transition we put it into replay memory but I might not necessarily update for that transition right I will put it into replay memory and pick one transition at random from the replay memory and I will update it right, since I am picking things at random from replay memory I do not have this correlation.

So now I might be updating for some state here right next time I might be updating for some other state there because it might be picked from a different trajectory and so on so forth right so I am I do this kind of random sampling from replay memory which is this is what where it is different from Lin's replay, Lin's replay was on trajectories here the replay is on samples right and why are we able to do this because of Q learning.

And because it is off policy, it is Q learning and I am using the max over the next state right because of this I am able to do this right because I do not, remember I reiterated many times when we did Q learning that you do not have to update along trajectories for Q learning and here is a case where you actually put it into practical use right because you are going to get this from replay memory so you do not have to.

So there is a second artifact that is introduced by doing the online updates so what is that, it is kind of a biased sampling of the input space right because it is going to depend on the actions you take suppose I decide to go left at some point right so the kind of features I will see will all have the player to the left of the screen right so that again increases the correlation between the successive inputs I am going to see.

It also skews the kind of samples i am going to get I am not going to look at parts of the state space which are to the where the agent is to the right of the screen right I will only look at parts of the state space where the agent is on the left of the screen right if you remember what is this input going to be for Atari, the pixels on the screen right, just looking at whatever is coming on the screen.

So I am going to see only some parts of the state space in a single trajectory right so essentially what will happen is only some portion of the state space gets their value functions updated the other portions of the state space do not so this can lead to some kind of unbalanced updates in the weight space right so to and this kind of a transition replay takes care of that also right so these are the three main difficulties which the paper lists out.

One is inefficient use and the other one is this correlated updates and the skewed sampling right all of this three can be addressed by using a replay memory and picking things from the, picking things at the transition level yeah, last what, last problem right, so see suppose I decide to move to the left at some point right so essentially then for the next few time steps right I will be only seeing the agent to the left of the screen right.

All the states with the agent to the right of the screen will all get ignored so the sampling I am going to make is strongly driven by the policy that I have taken, yeah but the whole screen is there but that is just one state right the screen with the agent here is one state the screen with agent here is a different state but what I will be seeing is only some subset of the states right so that subset is determined by their policies I am taking, the actions I am taking.

Therefore things can get really skewed up right so only one side of the screen or only one part of the state space will get updates the other parts of the state space does not get update so this can lead to some kind of a you know because this side is all incorrect values right and this side is all getting correct values but then they might get updated by the incorrect values here so things can get completely messed up.

So you really want the whole state space to get some kind of uniform updates so that you do not mess up the overall backup process, the other online methods also had this problem that is a good point, it did not really did not creep up in the empirical setting until we moved on to such a large state spaces, right so only when you have went on to such very large state spaces where you have a significant difference it became a problem.

It is not a problem with neural networks at all, it is just the problem of doing online updates I did not say it as a problem with neural networks, it is a problem with neural networks to the extent that the function approximator has now become so complex right so what you do in one part of the state space can very non deterministically affect what is happening in other part of the state space earlier when you are looking at linear function approximators things are a little bit more comprehensible right.

So now people just take stabs in the dark saying oh this might be what is wrong and then they try fixing it and if it works then they conclude that was what was wrong okay so that is essentially how you do work with this really really multilayer complex neural architectures you really do not know what is going wrong so you make hypotheses you try fixing this you really do science now right so you make a hypothesis you try fixing it and if it works great if it does not you go back and change your hypothesis and try something else right.

So that is essentially what you do right so good so any other questions on this sorry take advantage, if your φ function does then great right you know all about convolutional neural networks do they take advantage of symmetry, depends on the kind of symmetry they do not take advantage of all the symmetries that are there right so depends on what the φ network does so if you can actually start injecting ways of taking care of symmetry then great right.

So sorry, sorry no that is all, that black box there that φ is deep learning okay, any other, even in the function approximator where, the Q', I remember Q' is a one layer neural network right take the DQN and then you have your completely connected layer that is your Q' right multi layer is it only only one layer it is a one layer neural network.

So this is, the original DQN is 2 layer and the current version of DQN is 3 layers but it is a convolutional neural network so it is not layers as you normally understand it and this is a one layer network that takes the output of this and chugs off and it is not one network, it is 18 different one layer networks, I mean you are not you are not restricted to use a neural network there right, your Q' could come from anything.

As long as you can compute a gradient through it, well considering that Q' is actually a neural network so if you think of it, it is like a four layer neural network right I split it up as φ and Q' but if you think of it, it is actually a single box from the input to Q' so it is like a four layer neural network, the first three layers are of one type and the fourth layer is of a different type.

Right so I mean it you are just talking semantics and you can apply that deep to anything your thing so if you want to say oh we are only doing deep representation learning on top of that I am doing a one layer neural network for regression yes sure you are welcome to say that right, there is no I mean there is no standard nomenclature here good.

IIT Madras Production

Funded by

Department of Higher Education

Ministry of Human Resource Development

Government of India

www.nptel.ac.in

Copyrights Reserved